



The Lazy Developer

08

MODULE

It's complicated. When your project is getting advanced.

You need to start paying extra attention as your project gets larger. More files, more code, and more areas for things to break. The tips in this module will help you navigate these challenges.

Table of Contents

- 01. Managing complex apps and understand the best ways to test your changes. (10 min)
- 02. Commenting your code: The lifeline of complex projects. (10 min)
- 03. File structure and multiple apps: Keeping it all organised. (8 min)
- 04. Continuous Deployment & CI/CD (10 min)

Managing Complex Projects: Why Complexity Demands Better Practices

As we've already discussed, version control, using Github, and documentation are absolutely key to keeping your project organised and just generally they are good habits to always have in all the software projects you take on.

Why You Need Better Practices as Projects Grow

- More files and code mean more places for bugs to hide. A single update in one place can accidentally break another feature you weren't even thinking about.
- Collaboration becomes trickier. If you have teammates, or even if you're just revisiting your own work after some time, understanding what's changed (and why) is harder without a clear record.
- It's easy to lose track. As your app evolves, you want to be able to look back and understand the history of changes, not just for troubleshooting but also for planning new features.

Quick recap:

1. Create your documentation files (readme, changelog, buglist, etc)
2. Maintain a git in your software directory and tie it to a repo in Github
3. Work on small changes at a time. Breakup larger ones into smaller sections if you can.
4. Create versioning, and discuss with AI if this is something that should warrant a fix, a feature change, or major change.
5. Always keep your documentation updated after you make changes so you're always up to date.

Testing After Changes: Making Sure Nothing Broke

When your project is simple, it's easy to spot when something's not working where you make a change, refresh the app, and if it breaks, you see it right away. But as your app grows, it's no longer realistic to check every feature by hand every time you update something. Bugs can sneak in where you least expect them, and a small change in one area might accidentally break something else.

This is why testing is a critical part of working on more advanced projects. Testing helps you catch problems early, avoid surprises, and feel more confident as you make improvements. Even if you're not writing code yourself, you can use AI and tools like Cursor to help you think through, generate, and even automate your tests.

Why Test After Every Change?

- Prevent accidental breakage: Sometimes a change works in one spot, but causes issues somewhere else.
- Save time debugging: The earlier you catch an error, the easier it is to fix.

- Build confidence: Knowing your app is still working as expected lets you move faster on new features.

Types of Testing You Should Know

You don't need to be an expert in all forms of testing, but it helps to understand the basics:

- Manual Testing: This means running your app, clicking around, and making sure everything behaves as it should. This is what most people start with, and it's still valuable, especially for new features or visual changes.
- Automated Testing: These are scripts that check if your app works automatically. For example, a test might check connectivity to an endpoint after making some changes.
- Integration Testing: This checks if different parts of your app work together. For example, if your app saves data to a database, an integration test might check that the save-and-retrieve flow is still working.

What Should You Look For After Making Changes?

- Broken features: Did something that worked before stop working?
- Error messages: Are there any new errors or warnings in the console?
- Unexpected side effects: Did your change accidentally affect a different part of the app?
- Performance issues: Is anything running much slower than before?

Using AI to Help With Testing

AI can help you stay on top of testing, even if you aren't sure where to start. Here are some practical ways to get AI involved:

Prompt Examples for AI:

- Generating Test Cases:
"Cursor, what should I test after updating the user registration form?"
- Writing Automated Tests:
"Cursor, write a test to check that the 'Reset Password' feature still works."
- Checking for Breakage:
"Cursor, are there any parts of my app that could be affected by this change?"
"Cursor, review the last few commits and suggest areas I should manually test."
- Spotting Errors in the Code:
"Cursor, scan my code for obvious mistakes or potential bugs after my latest changes."

Real-World Example

Let's say you've just updated how your app saves user data. Here's how you might approach testing:

1. Ask AI for a checklist:
"Cursor, generate a checklist of what to test after changing how user data is saved."

2. Run through manual tests:
Log in, create a new user, edit their details, delete a user—make sure each action still works.
3. Automate where possible:
Use AI to generate or update automated test scripts that cover those user actions.
4. Review results and errors:
Check if any tests fail or if any errors pop up in your logs.

Quick Tips

- Test small changes as you go. Don't wait until everything is done.
- Automate repetitive checks. Let AI do the heavy lifting for things you test often.
- Don't be afraid to ask AI for help. Even just asking for "what could break?" is a smart move.

Commenting Your Code: The Lifeline for Complex Projects

As your app grows, the codebase gets more complicated. Even with AI's help, it can become tricky to remember why something was done a certain way, what each part is responsible for, or how different pieces fit together. This is where comments come in. They're notes you (and your future self, or your teammates) leave inside the code to explain what's happening, why you made certain decisions, and how everything works together.

Good commenting is one of the most underrated tools for preventing confusion, saving time, and making collaboration smooth even if you're the only one working on your app. It's like leaving breadcrumbs for anyone who needs to understand the project later, including yourself after a few months away.

Why Comments Matter (Especially As Projects Grow)

- **Easier debugging:** When something breaks, comments help you quickly find what each section does.
- **Smooth handovers:** If someone else ever works on your code, they can understand your thought process.
- **Future-proofing:** You won't have to "re-learn" your own logic when coming back to the project after a break.
- **Faster AI assistance:** Well-commented code helps tools like Cursor give you more accurate suggestions and code completions.

Commenting Strategies That Work

You don't have to comment every single line. Instead, focus on making your code clear where it needs it most:

- **Explain the "why," not just the "what":**
Don't just describe what the code does (that's often obvious). Focus on explaining why you did it this way, especially if it's not the most straightforward solution.
- **Comment tricky logic or workarounds:**
Any time you do something unusual, add a comment explaining the reason. For example:
`// Using this method to handle edge cases with empty input (see issue #123)`
- **Mark TODOs and FIXMEs:**
When you know something isn't finished or could be improved, add a clear comment so you (or someone else) can revisit it.
`// TODO: Refactor this section to handle multiple user roles`
- **Document important functions and files:**
At the top of files and major functions, include a quick summary of what they do.
`// This file handles all user authentication logic for the app.
// Includes login, logout, and password reset functionality.`
- **Keep comments updated:**
If you change the code, check if the comment still makes sense. Outdated comments can be more confusing than none at all.

Enforcing a Commenting Strategy in AI

AI lets you set up project “rules” to guide how you work. It’s a great idea to add a simple commenting strategy to your rules file so you (and any collaborators) stay on the same page.

Example Cursor Rules (or Claude.md) Entry:

```
# Project Rules
- Every major function or component must have a descriptive comment.
- All tricky logic must be explained with a 'why' comment.
- Use TODO and FIXME tags for unfinished or temporary code.
```

Using AI to Generate and Review Comments

AI can help you comment code faster and more thoroughly, even if you’re not sure what to say.

Prompt Examples for AI:

- Generate comments automatically:
“add comments to this file explaining what each function does.”
- Review existing comments:
“check my code for missing or unclear comments.”
- Explain a specific section:
“add a comment to this function explaining why it uses this approach.”
- Clean up or update comments:
“review and update all comments in this file to make sure they are accurate.”

Real-World Commenting Example

Suppose you have a function that calculates discounts for a shopping cart. A good comment explains the logic and any business rules:

```
# Calculates the total discount for a shopping cart.
# Applies 10% off for orders over $100, plus special holiday discounts.
# Uses workaround for rounding issues (see bug report #45).
def calculate_discount(cart):
    ...
```

Quick Tips for Commenting Success

- Don’t overdo it: Aim for clarity, not clutter.
- Be consistent: Use the same style throughout your project.
- Let AI help: AI is great at generating and suggesting useful comments so don’t hesitate to ask but just check it’s not going overboard.

Good commenting is like giving yourself and others a map through your project. The bigger and more complex the app, the more valuable those little explanations become.

File Structure and Multiple Apps: Keeping It All Organized

As your projects grow not just in code, but in scope and ambition - the way you organize your files becomes more and more important. A messy workspace can lead to confusion, lost work, and bugs that are hard to trace. On the other hand, a clean, logical folder structure will help you find what you need, collaborate smoothly, and set yourself up for adding new features (or even new apps) in the future.

If you start working on more than one app, things can get confusing quickly unless you're careful about separation from the start. The best approach is to keep each app in its own dedicated folder inside the /projects directory of your Cursor workspace. This way, everything for each app is neatly contained, Git tracking stays simple, and switching between projects is seamless.

Why File Structure Matters

- Reduces confusion: You know exactly where to find things (and where to put new code).
- Prevents accidental overwrites: Files for different features or apps stay separate.
- Makes teamwork easier: Others can quickly understand how your project is organized.
- Simplifies backups and deployments: You can zip, copy, or deploy just what you need.

Recommended Folder Structure for a Single App

Here's a simple, scalable example you can use as a template for organizing a single app:

```
/projects
  /my-app
    /client      # All front-end (user interface) code
    /server     # All back-end (API, business logic) code
    /sql        # Database queries, migrations, or backups
    /tests      # Automated and manual test scripts
    /docs       # Project documentation or guides
    README.md   # Overview of your app
    .git        # Git version control data (managed by Git)
```

Tip: You can add more folders as needed—like /scripts for utilities, or /assets for images and fonts.

Handling Multiple Apps

Let's say you're working on two or three different apps. Here's how your workspace might look:

```
/projects
  /shopping-cart-app
    ...all folders and files for this app...
  /blog-platform
    ...all folders and files for this app...
  /portfolio-website
    ...all folders and files for this app...
```

With this structure, each app is totally self-contained. You can open just one app at a time in AI, or switch between them easily without mixing up code, Git history, or dependencies.

Why each app needs its own folder:

- Git tracks changes separately: You don't get tangled commit histories or risk breaking another project by accident.
- Different dependencies: Each app can have its own configuration, libraries, and environment settings.
- Easy deployment and backups: Move, copy, or deploy one app at a time.

Real-World Example

Suppose you're developing a shopping cart app and a personal blog at the same time. You'd create:

- /projects/shopping-cart-app
- /projects/blog-platform

Inside each, you'd use the recommended folder structure above, so the client, server, and other files for one app never mix with the other.

Tips for Organizing and Maintaining Structure

- Be consistent: Use the same folder names and hierarchy in every project.
- Separate "client" and "server": This keeps your front-end and back-end code from getting mixed up.
- Keep SQL and backups out of your main code folders: Use a /sql or /backups folder so these important files are easy to find.
- Put docs and guides in a /docs folder: Makes onboarding new collaborators (or your future self) faster.
- Don't be afraid to reorganize: As your app grows, you can always refactor your folder structure. AI can help automate this!

Using AI to Help With File Structure

AI can take a lot of the manual work (and uncertainty) out of organizing your project. Here are some useful ways to prompt:

Prompt Examples for AI:

- Generate a folder structure:
“generate a recommended folder structure for a web app with separate client and server code.”
- Move files to the right place:
“move all my SQL files into a new /sql folder and update any code references.”
- Explain what’s where:
“give me an overview of the main files and folders in this app.”
- Help with multiple apps:
“set up a new folder for my second app in /projects and initialize it with Git.”
- Switch between apps:
“switch my workspace to work on the /projects/blog-platform app.”

Quick Checklist for Project Organization

- Each app is in its own folder inside /projects
- Each app has clear subfolders (client, server, sql, tests, docs, etc.)
- Git is initialized inside each app’s folder (not the parent /projects folder)
- No mixing of files between apps
- Documentation and backups are easy to find

Continuous Deployment & CI/CD: Automate Your Way to Smoother Projects

This section would definitely be something that's more advanced when your app gets really large, really complex, and would have more than one person doing the work to maintain it typically. As your app grows, the process of testing, building, and deploying new changes by hand becomes time-consuming and risky. That's where CI/CD comes in. These are best practices that even non-coders can benefit from—especially with AI to help you set things up.

CI/CD stands for Continuous Integration and Continuous Deployment (or Continuous Delivery). The main idea: every time you make a change, there's an automatic process that checks your work and, if all tests pass, safely updates your live app. This helps you catch problems early, deploy confidently, and focus on building features instead of fighting fires.

Why CI/CD Matters for Complex Projects

- Fewer mistakes: Automated checks catch bugs and errors before they reach users.
- Faster updates: You can deploy new features or bug fixes quickly, sometimes automatically within minutes.
- Peace of mind: You know your app is always in a working state, with every update tested and reviewed.

Key Parts of a Typical CI/CD Workflow

Here's what usually happens when you set up CI/CD for your app:

1. Push Changes: You make a change and push it to your Git repository (AI manages this for you).
2. Automated Tests Run: A CI/CD tool automatically runs all your tests to make sure nothing broke.
3. Build Step: The app is assembled, compiled, or otherwise prepared for deployment.
4. Deployment: If all tests pass, the CI/CD tool automatically puts the new version live (or prepares it for review).
5. Notifications: You get an email or message if something fails, so you can fix it before users notice.

How This Fits Into Your Workflow

You don't have to be a DevOps expert to get started. AI can help you write the configuration files, scripts, and tests needed for basic CI/CD. Here's how you might use it:

Prompt Examples for AI:

- Set up basic CI/CD:
“generate a GitHub Actions workflow to run my tests and deploy to Vercel after each push.”
- Automate tests on every commit:
“create a configuration so my app's test suite runs automatically whenever I push to Git.”

- Get a summary of CI/CD status:
“explain what happens in my current CI/CD pipeline.”
- Add deployment steps:
“add a step to my CI/CD workflow to back up the database before deploying.”

Real-World Example

Suppose your project is on GitHub, and you want to make sure tests run every time you update the code. You’d ask AI to:

1. Create a `.github/workflows/main.yml` file that defines what happens after each push (e.g., install dependencies, run tests, deploy if all is good).
2. Write or suggest tests for your most important features.
3. Explain how to connect your deployment platform (like Vercel, Netlify, or Heroku) to your repository.

Once set up, every time you make a change and push it, everything else happens automatically. If a test fails, deployment is paused until you fix it saving you from accidental bugs going live.

Getting Started with CI/CD

- Start simple: You don’t need to automate everything at once. Start with running tests automatically, then add deployment when you’re ready.
- Use templates: Most platforms (like GitHub Actions, GitLab CI, or others) have starter templates. AI can help you pick and customize one for your app.
- Ask for explanations: If you’re not sure what a workflow file does, ask AI to break it down in plain language.

Quick Tips

- Keep workflows versioned: CI/CD config files should live in your code repo so changes are tracked.
- Update tests as features change: Your automated tests should grow as your app does.
- Automate backups before deploys: Add steps to back up your data so nothing gets lost if something goes wrong.

CI/CD is like having a robot assistant—it catches problems, automates boring work, and lets you focus on building features. AI can guide you step by step, whether you’re writing your first workflow or maintaining a complex app.